Optimización y preparación para producción de PharmaSoft

Informe de la actividad autónoma de la sesión 13

Estudiante. Brandon Erly Pacho Mamani

Curso. Lenguaje de Programación II

Proyectos. PharmaBackend y pharma-frontend

Rama. feature/optimizacion-produccion-pacho

Fecha de mediciones. 8 de noviembre de 2026, hora de Lima

El informe reúne las comprobaciones del Reto 02 con la estructura de la práctica de S13. Cada ficha incluye procedimiento, resultado, estado y evidencias. Las mediciones de navegador provienen de dos rondas del mismo entorno local. La ronda inicial registró precarga, recarga y despliegue. La ronda en Edge instalado confirmó el analizador, la matriz de catálogos, la venta y los códigos de seguimiento.

Resumen de resultados

Comprobación	Resultado observado
Tamaño inicial	313,59 a 305,78 kB. Reducción de 2,49 %.
Redis	19 aciertos y 1 fallo de 20 consultas en cada catálogo. TTL de ensayo expirado y restaurado a 10 minutos.
Venta e invalidación	Venta de ensayo 141. Stock del producto 99 de 10 a 9. Seis casos de matriz verificados.
Seguimiento y salud	La venta se correlaciona con el identificador 4b4dd7c3. Redis detenido deja health en 503 y productos en 200.
Entrega	Informe y evidencias locales listos. El historial de tres fechas, los PR y la subida al aula se controlan en la parte F.

Entorno. Windows, Edge y Postman instalados, Java 21, Oracle local, Redis 7 y Nginx en Docker Desktop. Nginx sirve la SPA en 8081 y la API responde en 8080.

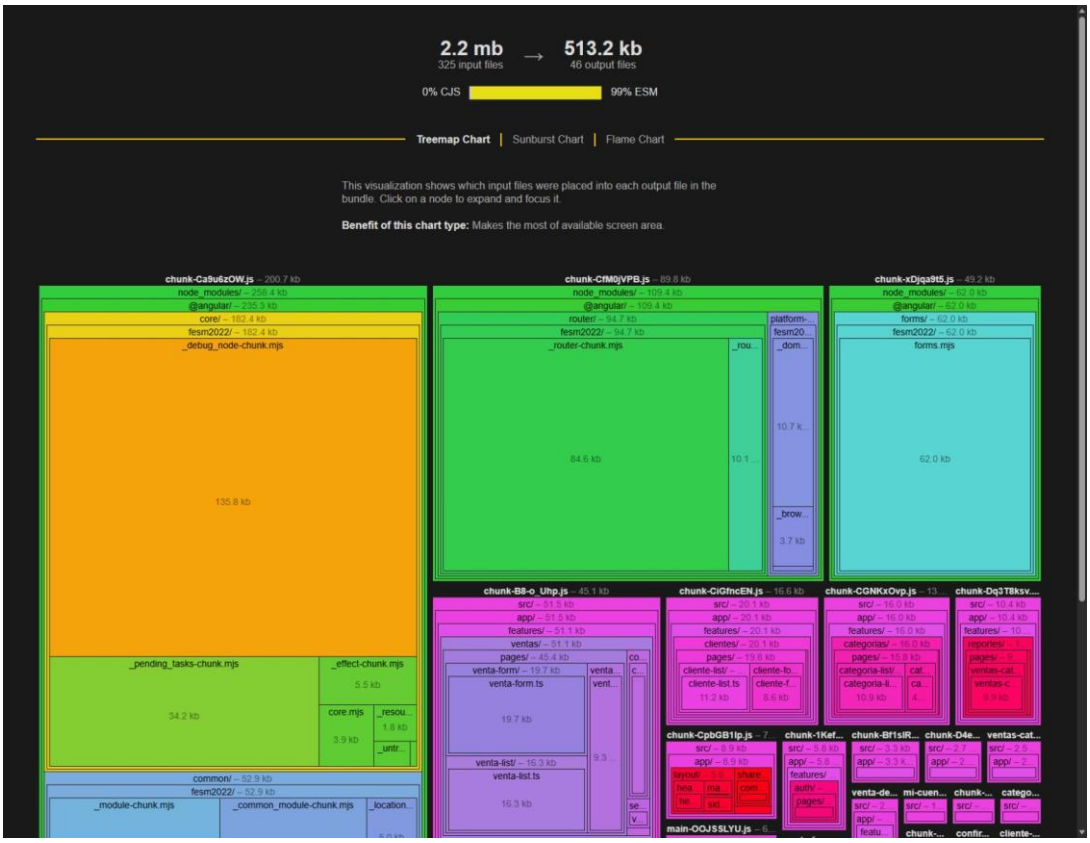
A01 Analizador y módulos iniciales

Procedimiento. Compilé Angular en producción con estadísticas e importé browser-stats.json en esbuild Bundle Size Analyzer desde Edge. Identifiqué los archivos iniciales del CLI y sumé bytesInOutput de sus imports estáticos. El gráfico general incluye también salidas diferidas.

Archivo fuente	Aporte inicial	Origen
@angular/core/fesm2022/_debug_node-chunk.mjs	139 085 B	Angular
@angular/router/fesm2022/_router-chunk.mjs	86 669 B	Angular
@angular/core/fesm2022/_pending_tasks-chunk.mjs	35 017 B	Angular
@angular/common/fesm2022/_module-chunk.mjs	25 087 B	Angular
@angular/common/fesm2022/_common_module-chunk.mjs	21 675 B	Angular

Resultado observado. Los cinco mayores aportes corresponden a Angular Core, Router y Common. La unidad visual del analizador usa escala binaria. La tabla presenta bytes exactos del metafile.

Figura 01. Metafile final en el analizador de esbuild abierto en Edge.



Estado. Conforme. Los detalles de cada módulo se conservan en AA-A01-04 a AA-A01-08.

A02 Comparación de tamaño antes y después

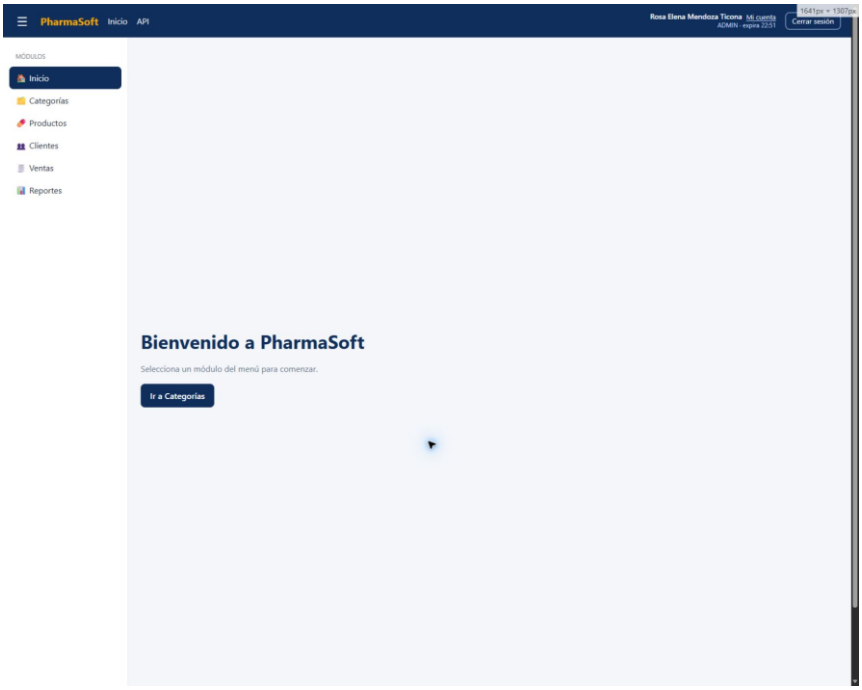
Procedimiento. Compilé dos veces con el mismo código y configuración de producción. Para el antes cargué MainLayout con un import estático. Para el después restauré loadComponent en la ruta protegida. Las salidas fueron a directorios de ensayo separados y la versión servida quedó intacta.

Medida	Antes	Después
Bytes iniciales	313 589	305 784
Tamaño inicial	313,59 kB	305,78 kB
Transferencia estimada del CLI	88,40 kB	86,32 kB
MainLayout	Incluido al inicio	Chunk diferido

Resultado observado. Se redujeron 7 805 bytes (2,49 %). La transferencia estimada del CLI se registra aparte. La comprobación en Edge abrió Inicio, Categorías, Productos y Ventas.

Evidencias. A02-build-antes.txt, A02-build-despues.txt y sus metafiles. A01-build-final.txt contiene los chunks servidos en la ronda Edge.

Figura 02. Inicio en Edge después de arrancar el backend en perfil prod.



Estado. Conforme.

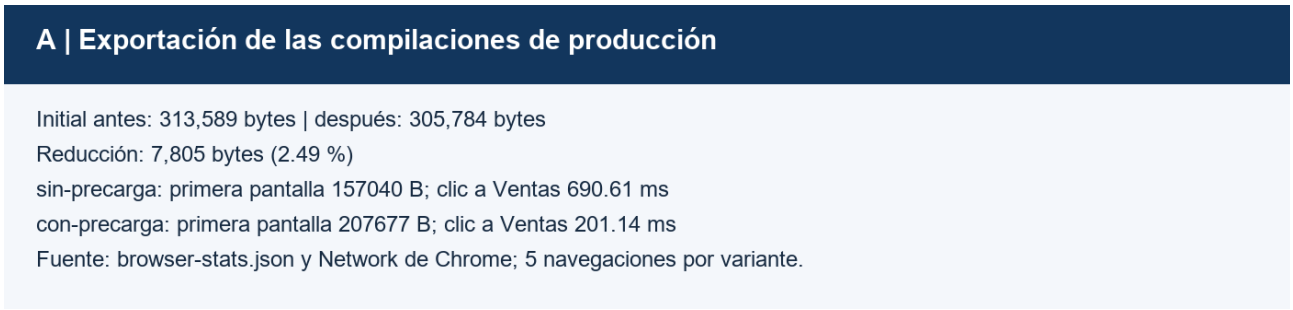
A03 Precarga de rutas

Procedimiento. La primera ronda comparó cinco contextos limpios por variante, con caché deshabilitada, 1,5 Mbit/s de bajada, 750 kbit/s de subida, 80 ms de latencia y una espera fija de 5 s tras el login. La medida hasta Ventas exigió el encabezado y el selector de clientes listo.

Configuración	Primera pantalla	Clic a Ventas	Login a Ventas
sin-precarga	157 040 B	690,61 ms	5994,24 ms
con-precarga	207 677 B	201,14 ms	5570,33 ms

Decisión. Se mantiene la descarga bajo demanda. La precarga transfirió 50 637 bytes adicionales durante la primera pantalla y redujo unos 489 ms entre el clic y la vista de Ventas. Para los tres locales, el ahorro de tráfico inicial justifica conservar la configuración actual. Se puede reevaluar con datos de uso de caja.

Figura 03. Resumen de datos capturados por Chrome durante la primera ronda.



Evidencias. A-sin-precarga.json y A-con-precarga.json contienen las cinco observaciones de cada variante. La cifra de login incluye la espera deliberada.

Estado. Conforme en la ronda de medición original.

B01 y B02 Cabeceras y origen en recarga

Procedimiento. Verifiqué los cinco recursos por HEAD con curl en Nginx. Para categorías usé una petición autenticada. Los orígenes corresponden a una recarga anterior registrada mediante el protocolo de DevTools. Los nombres con hash pertenecen a cada compilación.

Recurso comprobado	Cache-Control	Origen en recarga
/index.html	no-cache	Red y revalidación
/main-OOJSSLYU.js	public, max-age=31536000, immutable	Memoria
/chunk-B8-o_Uhp.js	public, max-age=31536000, immutable	Memoria
/styles-AAJM6OEI.css	public, max-age=31536000, immutable	Memoria
/api/v1/categorias	no-cache, no-store, max-age=0, must-revalidate	Red

Resultado observado. HTML con no-cache para revalidación. JS y CSS con hash reutilizados desde memoria en la recarga. El API conserva no-store, Pragma no-cache y Expires 0. Guardar respuestas autenticadas del API en el navegador podría exponer datos o mostrar precios y stock obsoletos.

Figura 04. Cabeceras del ensayo original con curl. La ronda final conserva sus cinco respuestas en B01-cabeceras-curl.json.

B | Exportación de curl -I en Nginx 8081

```
/index.html
  HTTP/1.1 200 OK | Cache-Control: no-cache
/main-D676CMNU.js
  HTTP/1.1 200 OK | Cache-Control: public, max-age=31536000, immutable
/chunk-CosgKqug.js
  HTTP/1.1 200 OK | Cache-Control: public, max-age=31536000, immutable
/styles-AAJM6OEI.css
  HTTP/1.1 200 OK | Cache-Control: public, max-age=31536000, immutable
/api/v1/categorias
  HTTP/1.1 200 | Cache-Control: no-cache, no-store, max-age=0, must-revalidate
```

Estado. Conforme. B-recarga.json registra red, memoria y disco con los indicadores de DevTools.

B03 Despliegue con Nginx activo

Procedimiento. En la primera ronda conservé la pestaña de Productos, cambié el título visible y reconstruí la SPA en el volumen que Nginx ya estaba sirviendo. Recargué la misma pestaña sin vaciar su caché.

Resultado observado. El título nuevo apareció. Cambiaron 12 nombres de archivos y main pasó de main-D676CMNU.js a main-HWAKLLAQ.js. styles-AAJM6OEI.css conservó su nombre. B-despliegue.json identifica los archivos que el navegador solicitó y los que reutilizó. El HTML revalidado refirió la nueva cadena de archivos.

Figura 05. Productos antes de reconstruir la SPA.



Figura 06. Nuevo título en el mismo navegador después de recargar.



Estado. Conforme en la ronda original. En producción se recomienda activar la versión en forma atómica y conservar los chunks antiguos durante sesiones abiertas.

C01 a C03 Aciertos tiempos y expiración

Procedimiento. Eliminé únicamente la clave de ensayo de cada catálogo. Registré contadores hit y miss antes y después de 20 GET secuenciales con los mismos parámetros. El cliente HTTP midió el tiempo total de cada petición y el resultado se calculó con deltas de Actuator.

Caché	Aciertos	Fallos	Tasa	Primer GET	Media de 19 aciertos
categorias	19	1	95 %	23,733 ms	20,601 ms
productos	19	1	95 %	28,947 ms	24,240 ms

Interpretación. 19 / (19 + 1) da 95 %. La primera consulta es una observación única del fallo. Los tiempos incluyen autorización, transporte y procesamiento. Redis evita consultar el catálogo en Oracle cuando hay acierto.

Configuración y acción	TTL observado
CACHE_TTL=1m y una consulta de la clave nueva	60 segundos
Después de 64 s sin volver a consultar la clave	-2 (clave inexistente)
CACHE_TTL=10m y consulta posterior al reinicio	600 segundos

Evidencias. C01-C02-series-20.json conserva las 40 consultas y contadores filtrados. C03-ttl-inicial.json, C03-ttl-expirada.json y C03-ttl-restaurada.json conservan hora, clave y salida numérica de redis-cli.

Figura 07. Registro visual de la primera ronda. Los tiempos de la tabla corresponden a la repetición final.

C | Exportación de las 20 consultas por caché

categorias: 19 hit / 1 miss = 95 %; primero 499.175 ms; media hit 29.214 ms
productos: 19 hit / 1 miss = 95 %; primero 83.694 ms; media hit 28.172 ms
redis-cli TTL categorias::pagina-0-7-nombre-asc -> 60 | 2026-11-09T00:58:22.453Z
redis-cli TTL categorias::pagina-0-7-nombre-asc -> -2 | 2026-11-09T01:02:25.748Z
redis-cli TTL categorias::pagina-0-7-nombre-asc -> 600 | 2026-11-09T01:02:49.894Z
Fuente: Actuator, cliente HTTP y redis-cli del entorno de ensayo.

Estado. Conforme. El backend quedó con TTL de 10 minutos.

C Matriz de invalidación

Caso	Operación	Resultado observado	Estado
C-01	Crear categoría	La categoría 147 apareció en la misma lista activa.	Conforme
C-02	Renombrar categoría	El producto mostró el nombre nuevo inmediatamente.	Conforme
C-03	Editar precio	S/ 2,50 a S/ 3,75 en lista y detalle.	Conforme
C-04	Registrar venta	Venta 141 desde la SPA. Stock 10 a 9.	Conforme
C-05	Dar de baja categoría vacía	La categoría 148 desapareció de activas. El historial conserva Inactivo.	Conforme tras corrección
C-06	Detener Redis	Productos 200 y health 503. Recuperación a UP.	Conforme

Procedimiento. Consulté las listas antes de escribir y repetí los mismos filtros tras cada operación. Usé registros de ensayo y verifiqué precio y stock del mismo producto. La baja conserva un registro de historial y la vista habitual filtra en el servidor antes de paginar.

Hallazgo C-05. La versión anterior mostraba las categorías inactivas en la lista habitual. Se agregó soloActivas a la consulta y a su clave de caché. La SPA activa ese filtro al abrir Categorías y permite consultar el historial con Mostrar inactivas.

Figura 08. C-01. Lista activa antes de crear la categoría de ensayo.

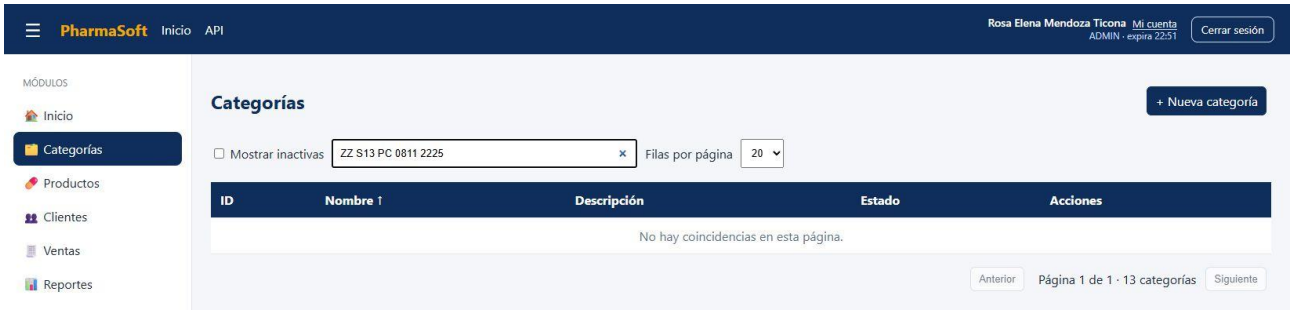
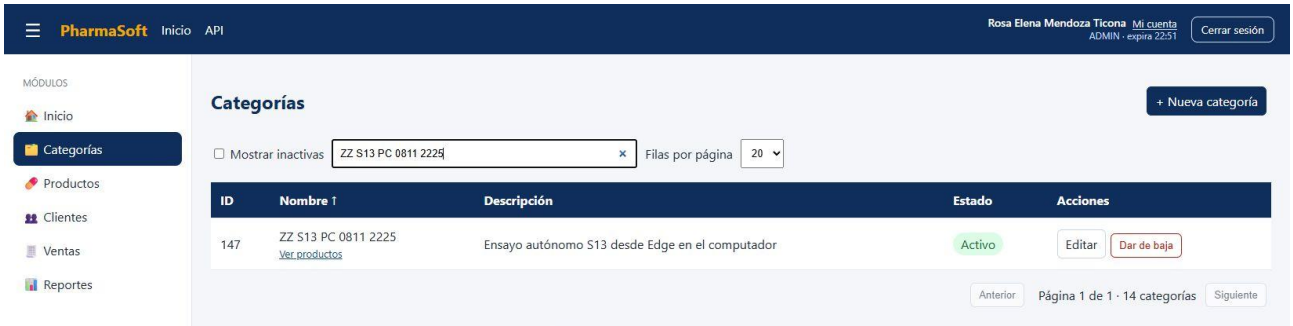


Figura 09. C-01. La categoría 147 aparece tras la creación.



C Evidencias de renombrado y edición

C-02 observado. La fila del producto 99 pasó de ZZ S13 PC 0811 2225 a ZZ S13 PC 0811 2225 nueva en la columna Categoría.

Figura 10. C-02. Columna Categoría antes de renombrar.



Figura 11. C-02. La misma fila muestra el nombre de categoría nuevo.



C-03 observado. La edición y el GET de detalle devuelven 3,75. La lista muestra el mismo precio. La captura anterior de C-02 muestra 2,50 y sirve como línea base de la lista.

Figura 12. C-03. Precio actualizado en la lista, con stock inicial de 10.



Evidencias adicionales. AA-C06-01, AA-C06-03 y AA-C06-04 muestran el formulario de detalle antes, al editar y después. C-matriz-api.json registra detalle y lista.

C Evidencias de venta y stock

Procedimiento. Vendí una unidad del producto 99 a un cliente ficticio mediante el formulario de Edge. Dejé la venta de ensayo en el historial. Comparé la misma fila de Productos antes y después.

Figura 13. C-04. Revisión de una unidad con total de S/ 3,75.



Figura 14. C-04. Detalle del comprobante de la venta de ensayo.

Producto	Precio	Cant.	Subtotal
ZZ S13 PC 0811 2225 producto	S/ 3.75	1	S/ 3.75
1 producto · 1 unidad		**Total**	**S/ 3.75**

Figura 15. C-04. El mismo producto queda con 9 unidades tras la venta 141.



Estado. Conforme. La figura de precio anterior muestra 10 unidades y C-matriz-api.json confirma el stock posterior.

C Evidencias de baja y recuperación

Figura 16. C-05. Categoría vacía activa antes de la baja.

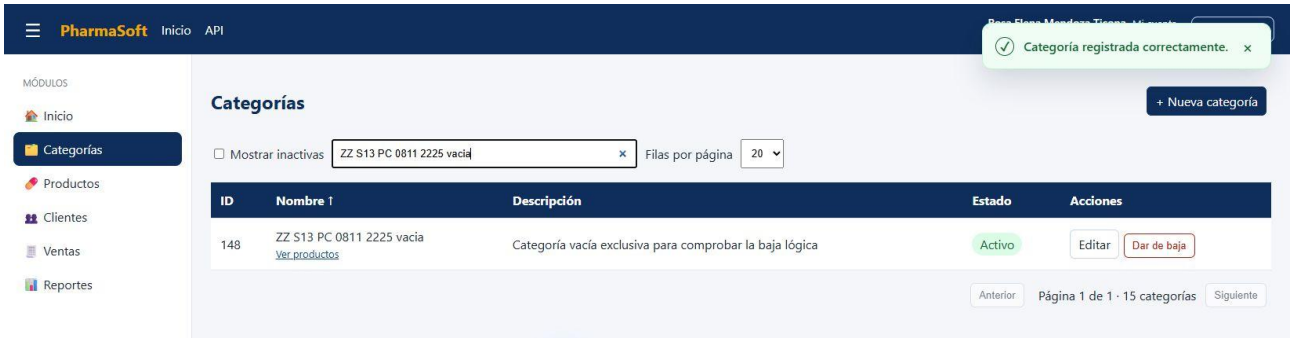
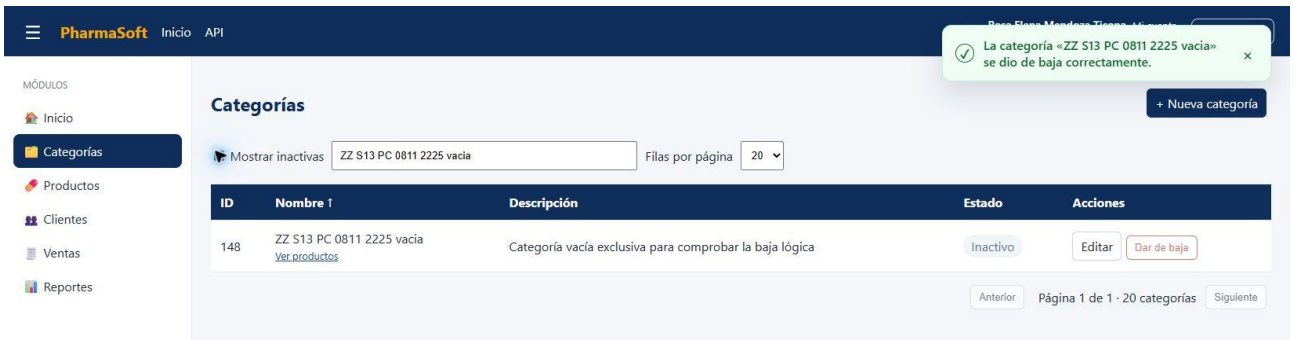


Figura 17. C-05. La búsqueda queda sin coincidencias en activas.



Figura 18. C-05. Mostrar inactivas permite consultar la misma categoría.



C-06 observado. Con Redis detenido, productos respondió 200 y health 503. El primer sondeo posterior a PONG seguía en DOWN. El sondeo posterior confirmó UP. La reconexión de la aplicación se comprobó de forma independiente del ping de Redis.

Estado. Conforme. En D04 se conservan los cuatro estados de health y la evidencia de continuidad.

D01 Trazabilidad de la venta

Procedimiento. Registré la venta desde Edge y encontré el identificador del POST con el ID interno de venta. Filtré el log de producción por ese identificador y conservé las tres líneas. La primera ronda también registró X-Request-Id directamente en la respuesta de la SPA para su venta de ensayo.

```
2026-11-08 22:27:14,647 INFO  [4b4dd7c3] p.c.u.P.s.impl.VentaServiceImpl - Inicio registrar venta | lineas=1
2026-11-08 22:27:14,807 INFO  [4b4dd7c3] p.c.u.P.s.impl.VentaServiceImpl - Venta persistida | ventaId=141 | lineas=1
2026-11-08 22:27:14,842 INFO  [4b4dd7c3] p.c.u.P.config.RequestIdFilter - POST /api/v1/ventas -> 201 (349 ms)
```

Línea	Interpretación
Inicio registrar venta	Comienza el procesamiento de una línea de venta.
Venta persistida	Oracle asignó el ID 141. El evento se registra dentro de la transacción.
POST 201	La petición terminó con creación exitosa. El filtro registró 349 ms.

Evidencia de cabecera. D-venta-spa.json de la primera ronda conserva el código 201 y su X-Request-Id. D01-identificadores.json y D01-traza-venta.txt corresponden a la venta 141 de Edge. Cada venta conserva su propio identificador.

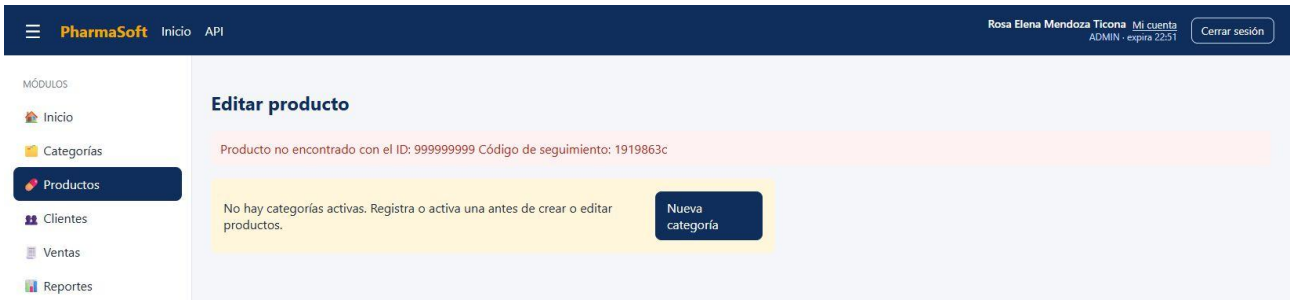
Estado. Conforme con la evidencia conjunta de ambas rondas.

D02 Errores con código de seguimiento

Prueba	HTTP	requestId del cuerpo	Coincide con la cabecera
noEncontrado	404	577be00e	Sí
sinToken	401	51f92dbd	Sí
validacion	400	d77432d5	Sí
stockInsuficiente	409	b1376900	Sí

Resultado en Edge. Producto no encontrado con el ID: 999999999 Código de seguimiento: 1919863c

Figura 19. Error de producto inexistente y código de seguimiento en la interfaz.



Implementación. RequestIdFilter genera el identificador y lo coloca en la cabecera y en el contexto de log. GlobalExceptionHandler lo agrega al cuerpo y los filtros de seguridad lo devuelven para errores de acceso. La SPA usa el cuerpo y, como alternativa, la cabecera.

Incidencia de despliegue. Nginx resolvía el alias host-gateway con una IPv6 inalcanzable y podía devolver 502. Cambié el destino local a host.docker.internal en Docker Desktop, validé nginx -t y recargué. Ocho peticiones seguidas alcanzaron la API y devolvieron 401 sin token. La repetición autenticada en Edge mostró el 404 esperado.

Estado. Conforme después de corregir la ruta del proxy local.

D03 Auditoría del log de producción

Procedimiento. Audité 158 líneas de la ronda de producción y busqué patrones de credenciales y DNI. Comprobé también la contraseña de ensayo en memoria y guardé solo el conteo.

Comprobación	Coincidencias
Palabras password, Bearer o secret	0
Patrón de JWT completo	0
DNI explícito completo	0
Contraseña ADMIN de ensayo	0
Clave de firma del arranque auditado	0

Hallazgo y corrección. ClienteServiceImpl escribía el DNI completo al registrar un cliente. El evento de inicio ahora omite ese dato y el evento de éxito conserva el identificador interno. Los logs históricos requieren retención y acceso restringido.

Clave JWT. En un nuevo arranque con perfil prod y TTL de 10 minutos, busqué la clave de firma exacta directamente en memoria. El backend llegó a UP y se revisaron tres archivos de registro, con 910 líneas en total.

Estado. Conforme en el alcance registrado en D03-auditoria.json y D03-clave-jwt.json. Ambos archivos guardan únicamente los conteos de coincidencias.

D04 Salud del sistema y métricas de ventas

Redis	Sin token	Con ADMIN	API Productos
Encendido	200 UP, sin componentes	200 UP, db y redis UP	200
Apagado	503 DOWN, sin componentes	503 DOWN, redis DOWN	200
Restaurado	200 UP tras reconectar	200 UP, redis UP	200

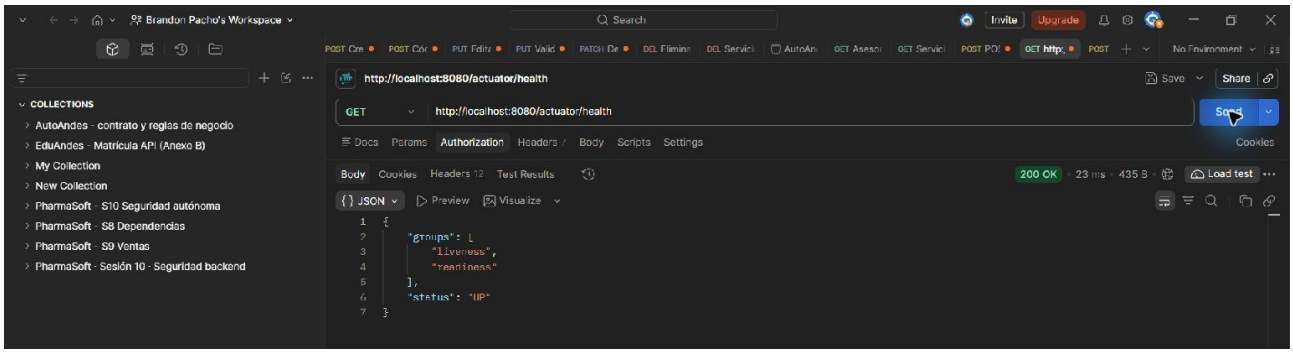
Métrica filtrada por uri /api/v1/ventas. COUNT=2. TOTAL_TIME=0,3691234 s. Media=184,562 ms. MAX de la ventana=20,017 ms. La muestra reúne el POST exitoso 201 y el rechazo 409 por stock insuficiente.

Interpretación. La respuesta de salud global se degrada si Redis falla, mientras productos puede seguir consultando Oracle. El máximo de Actuator corresponde a la ventana del medidor y puede haber caducado la observación más lenta. El promedio es descriptivo de dos solicitudes.

Figura 20. Productos en la SPA durante la caída de Redis, captura de la ronda original.



Figura 21. Consulta pública de salud en Postman, con Redis activo.



Evidencias. C09-health-off.json, C09-health-on-transicion.json, C09-health-on.json y D04-metricas-ventas.json.

Estado. Conforme.

E Despliegue en producción

Los tres locales acceden al mismo Nginx. PharmaBackend consulta Oracle para los datos y Redis para los catálogos. En producción se propone exponer Nginx por HTTPS y mantener las dependencias en la red privada.

```
Navegadores de tres locales → Nginx HTTPS 443 → PharmaBackend 8080
                                                ├─ Oracle 1521
                                                └─ Redis 6379
```

Servicio	Laboratorio	Producción propuesta
Nginx	Host 8081 → contenedor 80	443 público y 80 para redirección
PharmaBackend	8080	8080 privado
Oracle	Host 1522 → contenedor 1521	1521 privado
Redis	127.0.0.1:6379	6379 privado y compartido

Configuración	Variables y propósito
Backend y Oracle	SPRING_PROFILES_ACTIVE, SERVER_PORT, DB_URL, DB_USERNAME y DB_PASSWORD seleccionan perfil, puerto y conexión.
Autenticación y orígenes	JWT_SECRET, JWT_EXPIRACION_MINUTOS y FRONTEND_ORIGINS definen firma, vigencia y orígenes permitidos.
Redis y migraciones	REDIS_HOST, REDIS_PORT, CACHE_TTL, opciones SPRING_DATA_REDIS_* y SPRING_FLYWAY_ENABLED configuran caché y esquema.

Construcción y reversión. Compilar y validar el JAR con Java 21 y el frontend con npm. Respaldar Oracle antes de migrar. Publicar artefactos versionados y activar index.html cuando sus hashes estén disponibles. Probar login, salud, catálogo y venta. Si falla, reactivar JAR y SPA compatibles y aplicar la reversión de datos prevista para la migración.

Evidencia. La sección Despliegue en producción de ambos README contiene el diagrama y el procedimiento completo.

E Estado de prácticas y recomendaciones

Práctica	Estado	Acción
Carga diferida y tamaños	Cumple	Vigilar presupuesto de entrega en nuevos cambios.
HTML revalidado y JS/CSS con hash	Cumple	Conservar hashes de la versión anterior durante las sesiones.
Cache-Control del API	Cumple	Conservar no-store al desplegar el proxy.
TTL, invalidación y contingencia Redis	Parcial	Tratar escrituras durante la caída y concurrencia de repoblación.
Logs y Actuator	Cumple en laboratorio	Centralizar logs y configurar alertas de producción.
TLS y restauración de Oracle	Pendiente	Configurar HTTPS y ensayar recuperación antes de abrir los locales.

Prioridad 1. Habilitar HTTPS en Nginx, proteger API y bases en red privada y usar cuentas de producción con privilegios acotados.

Prioridad 2. Programar copias cifradas de Oracle y ensayar restauración. Medir el tiempo real de recuperación antes de definir un compromiso para los locales.

Prioridad 3. Compartir Redis entre instancias y alertar ante health degradado, errores y latencia. Documentar la purga selectiva de claves afectadas por escrituras durante una caída.

F01 Verificación de los commits de entrega

Procedimiento. Exporté los archivos registrados de cada rama a carpetas independientes con git archive. En frontend reutilicé las dependencias instaladas. En backend ejecuté Maven con perfil prod, Oracle y Redis locales, Flyway deshabilitado y validación del esquema.

Versión evaluada	Comprobación	Resultado
Frontend f0a0bd7	npm test en copia del commit	43 aprobadas, 2 fallidas de 45
Frontend 66b0064	Repetición tras corregir las dos pruebas	45 aprobadas, 0 fallidas
Backend cf2dc09	Suite completa de Maven, incluida la prueba de contexto	43 aprobadas, 0 fallos, 0 errores y 0 omitidas

Hallazgo y corrección. Las pruebas de Inicio y CategoriaForm fallaban al crear el componente porque su entorno no ofrecía ActivatedRoute. Incorporé provideRouter([]) a ambos módulos de prueba. El commit conserva las 45 pruebas y la repetición completa aprobó las dos que habían fallado.

Alcance. La ronda anterior obtuvo 53 pruebas aprobadas en el directorio de trabajo. F01 valida los archivos registrados en la rama y su compilación registró 295,84 kB de tamaño inicial. Angular conservó un aviso de 724 bytes sobre el presupuesto de CSS del formulario de venta.

Comprobación HTTP	Resultado observado
Ruta /ventas	200 · HTML · no-cache
Archivo JS inicial	200 · nombre con hash · immutable
API de categorías sin sesión	401 · requestId 342b87f1 coincide en cabecera y cuerpo
Salud del backend	200 · UP

Evidencias. La compilación del commit se sirvió en un Nginx temporal en localhost:18081 y se verificó con curl. La carpeta de la parte F contiene las tres salidas de pruebas, la compilación y F01-http-nginx.json.

Captura de navegador. Pendiente de adjuntar la pestaña Red de Chrome. La tabla conserva la verificación de transporte ejecutada sobre el entorno aislado.

F02 Estado de versionamiento y entrega

Repositorio	Fechas de la rama S13	Ramas en el remoto al verificar
PharmaBackend	2026-11-06, 2026-11-08	Rama de trabajo en 1a79813. develop pendiente de publicar.
pharma-frontend	2026-11-08	develop en c72b6e1. Rama de trabajo pendiente de publicar.

Historial. El conteo usa git log develop..HEAD y fechas reales de autor. Hay dos fechas en backend y una en frontend. Los commits adicionales del 8 de noviembre aportan evidencia de trabajo de esa jornada.

Solicitudes hacia develop. Los textos de las solicitudes están preparados. El envío depende de publicar la rama de trabajo de cada repositorio y de contar con la rama destino remota del backend.

Aula virtual. La copia Pacho_LP2_S13_Autonoma.pdf queda preparada para adjuntar. El comprobante de entrega está pendiente.

Reproducción. F02-historial-PharmaBackend.txt, F02-historial-pharma-frontend.txt y F-resumen.json conservan el historial y la consulta de ramas. La guía editable describe los pasos A01 a F02.

Fuentes. Guía de la actividad autónoma S13 de la UPeU. Documentación oficial de Angular CLI, esbuild Bundle Size Analyzer, Spring Boot Actuator, Redis TTL y Nginx proxy_pass.

https://angular.dev/tools/cli/build

https://esbuild.github.io/analyze/

https://docs.spring.io/spring-boot/reference/actuator/metrics.html

https://redis.io/docs/latest/commands/ttl/

https://nginx.org/en/docs/http/ngx_http_proxy_module.html#proxy_pass